Trends Application Deployment

DevOps Project — End-to-End CI/CD Pipeline

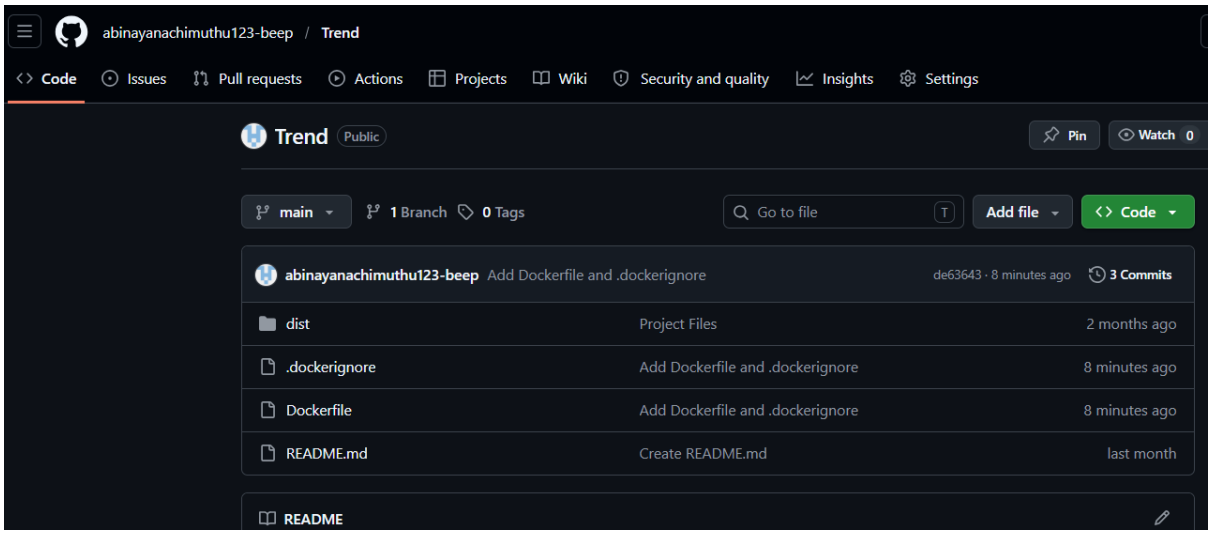
Field	Details
GitHub	https://github.com/abinayanachimuthu123-beep/Trend
Docker Hub	abinayacould/trend-app:latest
Jenkins	http://44.242.198.41:8080
EKS Cluster	trend-cluster — us-west-2
LoadBalancer	a5ca4eded6a844b8c8fcc94d562d7975-374557975.us-west-2.elb.amazonaws.com

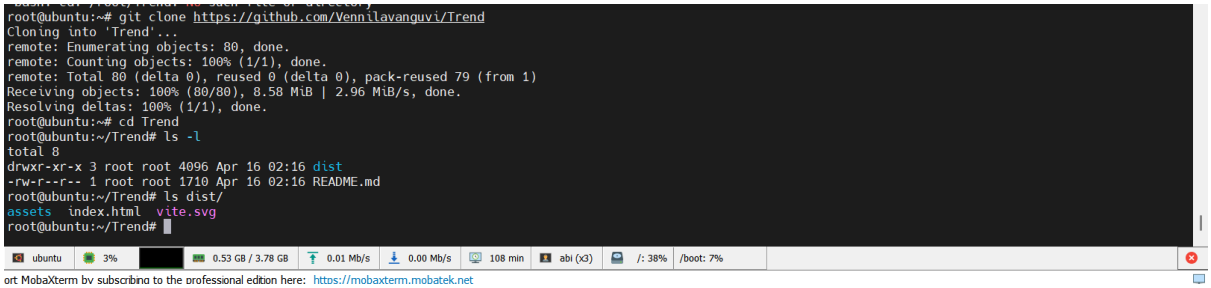
Part 1 — Application Setup

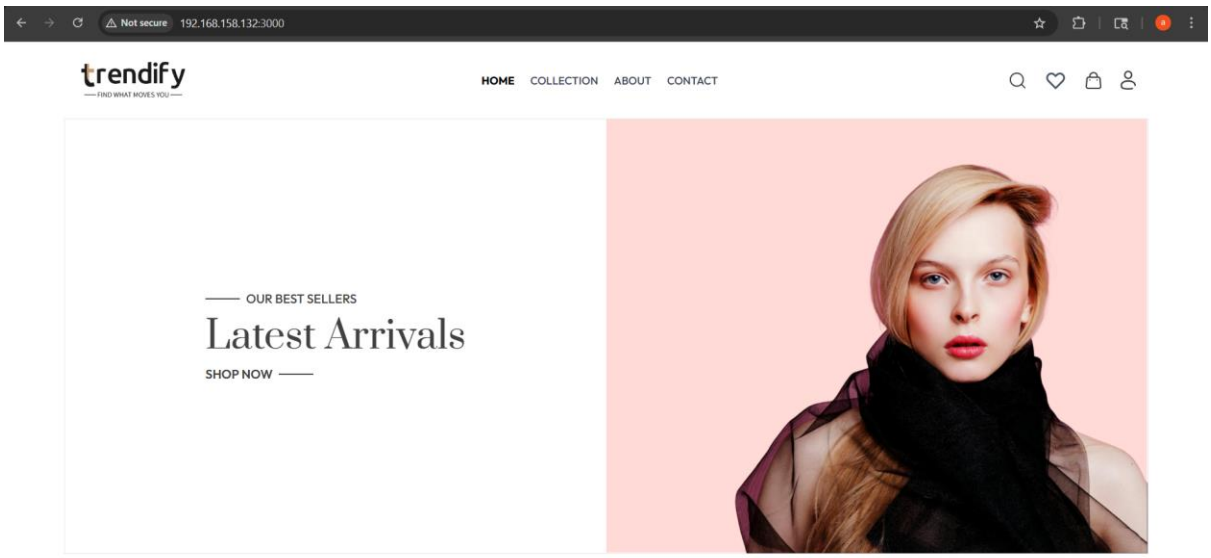
The React application was cloned from the source repository into the student's GitHub account and cloned onto a fresh Ubuntu VM. The app was served on port 3000 using the serve package pointing to the pre-built dist/ directory.

Steps completed:

- Cloned repo to Ubuntu VM
- Installed Node.js 18 and serve package
- Served app on port 3000 — accessible from host machine







Part 2 — Docker

The application was containerized using Docker with an nginx:alpine base image. The production dist/ folder is served by nginx inside the container. The image was built, tested locally, and pushed to Docker Hub.

Steps completed:

- Dockerfile created using nginx:alpine base image
- Docker image built — tagged as trend-app
- Container tested locally on port 3000
- App verified accessible in browser via container
- Image tagged and pushed to Docker Hub — abinayacould/trend-app:latest

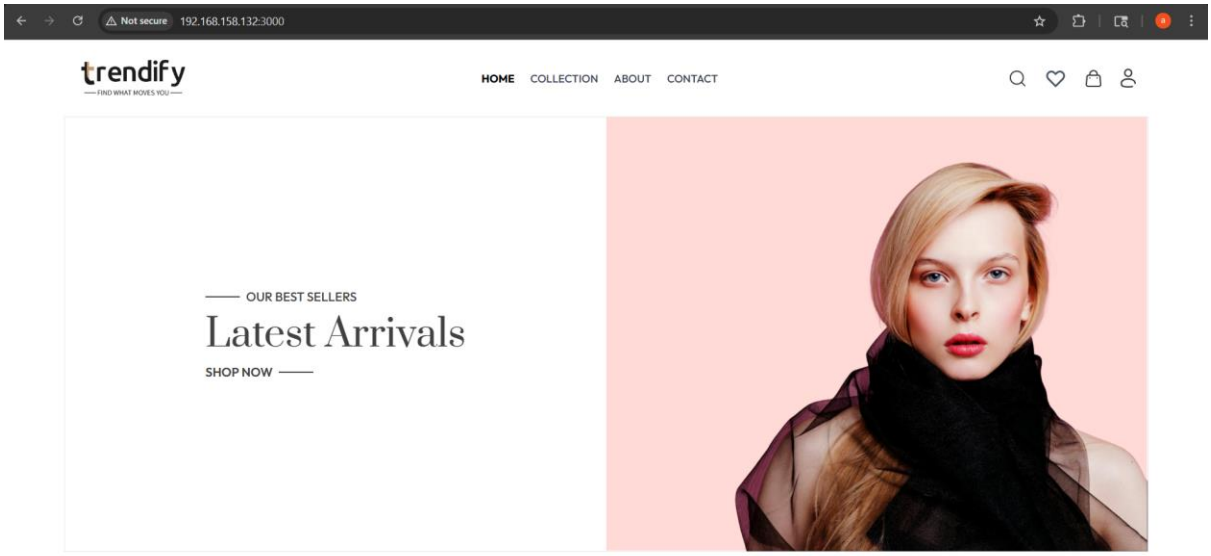
Container running on port 3000

App accessible via browser
Image pushed to abinayacould/trend-app:latest on Docker Hub

```
total 4
drwxr-xr-x 4 root root 4096 Apr 12 05:48 Trend
root@ubuntu:~# cd Trend
root@ubuntu:~/Trend# nano Dockerfile
root@ubuntu:~/Trend# cat Dockerfile
FROM nginx:alpine
COPY dist/ /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

```
COPY dist/ /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
root@ubuntu:~/Trend# docker build -t trend-app .
[+] Building 16.7s (7/7) FINISHED                                                                                docker:default
 => [internal] load build definition from Dockerfile                                                                      0.1s
 => => transferring dockerfile: 133B                                                                                      0.0s
 => [internal] load metadata for docker.io/library/nginx:alpine                                                           4.7s
 => [internal] load .dockerignore                                                                                         0.1s
 => => transferring context: 2B                                                                                           0.0s
 => [internal] load build context                                                                                         0.8s
 => => transferring context: 9.24MB                                                                                       0.8s
 => [1/2] FROM docker.io/library/nginx:alpine@sha256:582c496ccf79d8aa6f8203a79d32aaf7ffd8b13362c60a701a2f9ac64886c93d      9.9s
 => => resolve docker.io/library/nginx:alpine@sha256:582c496ccf79d8aa6f8203a79d32aaf7ffd8b13362c60a701a2f9ac64886c93d      0.2s
 => => sha256:34dfdd2ef1f920d0054dde2fc09ddc83ff8e71d05fadb79e2cab6e6234596f0a 1.21kB / 1.21kB                            0.9s
 => => sha256:1165b869c51a1a0747d78cec8fab96c30156a979e51ecf2f91aa792e557d94a4 20.25MB / 20.25MB                          6.3s
 => => sha256:c8a2fa3a88d244a3f32dcbc9c1f7649c662661a28c624198ada43aa0b7598e7f 1.40kB / 1.40kB                            1.9s
 => => sha256:ff9f59a6a62e9e9f29d7a84fb18865b45664d3f0d061eff7548bd61746dd101c 957B / 957B                                1.9s
 => => sha256:a71873b303e8d75170b7ced2725b01b3ae15ad76f0d4eef16a49335821b6a0ef 404B / 404B                                1.1s
 => => sha256:f03becc8ac15611cfcc421c977a5ba4d65456093570788523a4ba557689aa7f7 1.05MB / 1.87MB                            9.0s
 => => sha256:15e759724ff67f262e38bb7c070af9d0b84f959f9b37fa966f68bf2f881a4b62 627B / 627B                                1.1s
 => => sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153 3.86MB / 3.86MB                            4.9s
 => => extracting sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153                                 0.3s
 => => extracting sha256:f03becc8ac15611cfcc421c977a5ba4d65456093570788523a4ba557689aa7f7                                 0.3s
 => => extracting sha256:15e759724ff67f262e38bb7c070af9d0b84f959f9b37fa966f68bf2f881a4b62                                 0.0s
 => => extracting sha256:ff9f59a6a62e9e9f29d7a84fb18865b45664d3f0d061eff7548bd61746dd101c                                 0.0s
 => => extracting sha256:a71873b303e8d75170b7ced2725b01b3ae15ad76f0d4eef16a49335821b6a0ef                                 0.0s
 => => extracting sha256:34dfdd2ef1f920d0054dde2fc09ddc83ff8e71d05fadb79e2cab6e6234596f0a                                 0.0s
 => => extracting sha256:c8a2fa3a88d244a3f32dcbc9c1f7649c662661a28c624198ada43aa0b7598e7f                                 0.0s
 => => extracting sha256:1165b869c51a1a0747d78cec8fab96c30156a979e51ecf2f91aa792e557d94a4                                 1.3s
 => [2/2] COPY dist/ /usr/share/nginx/html                                                                                0.4s
 => exporting to image                                                                                                    1.3s
 => => exporting layers                                                                                                   0.9s
 => => exporting manifest sha256:a20b1264215b8116df395722c86ca48df9176e04923fa4085c61505947f22119                         0.0s
 => => exporting config sha256:f72c83f122ab02aa7087821e37f285e9462777885e39a6967dce681c09a29d0b                           0.0s
 => => exporting attestation manifest sha256:863276da11d8b7940e9747250416ba548ca50930b100d58c2fd77d7b6835837b             0.0s
 => => exporting manifest list sha256:079dd9240f80f4db722c2de060178839bc99acdf33aa192a129f9d2fdb7b1c6e                    0.0s
 => => naming to docker.io/library/trend-app:latest                                                                       0.0s
 => => unpacking to docker.io/library/trend-app:latest                                                                    0.2s
```

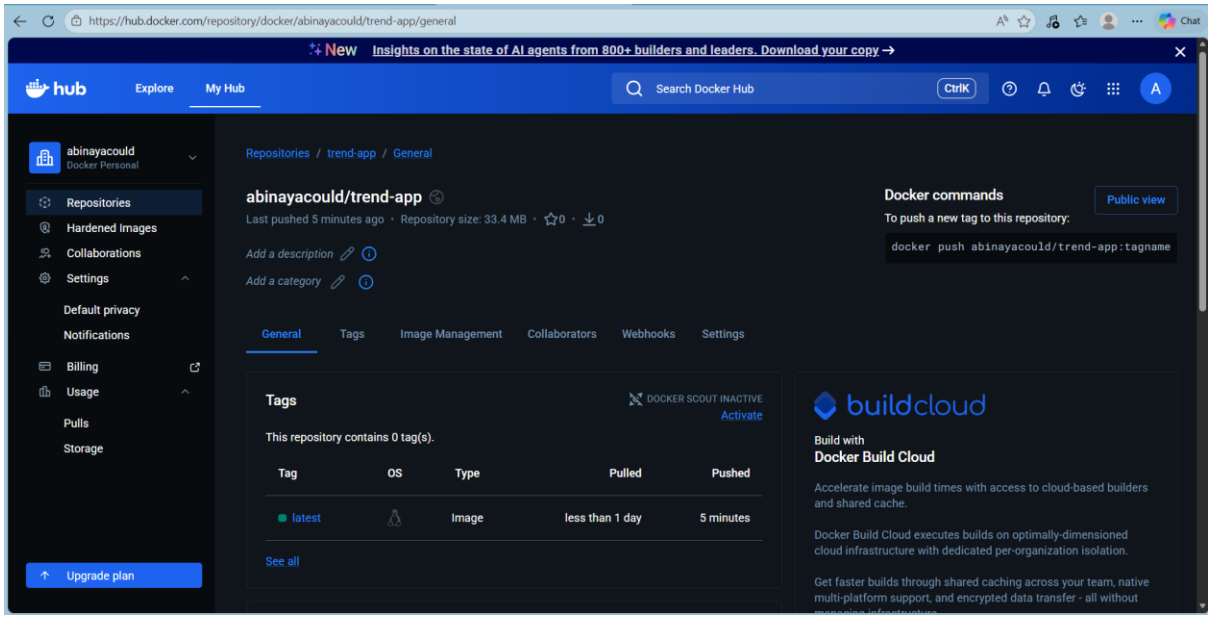
```
 => => sha256:34dfdd2ef1f920d0054dde2fc09ddc83ff8e71d05fadb79e2cab6e6234596f0a 1.21kB / 1.21kB                                    0.9s
 => => sha256:1165b869c51a1a0747d78cec8fab96c30156a979e51ecf2f91aa792e557d94a4 20.25MB / 20.25MB                                 6.3s
 => => sha256:c8a2fa3a88d244a3f32dcbc9c1f7649c662661a28c624198ada43aa0b7598e7f 1.40kB / 1.40kB                                    1.9s
 => => sha256:ff9f59a6a62e9e9f29d7a84fb18865b45664d3f0d061eff7548bd61746dd101c 957B / 957B                                        1.9s
 => => sha256:a71873b303e8d75170b7ced2725b01b3ae15ad76f0d4eef16a49335821b6a0ef 404B / 404B                                        1.1s
 => => sha256:f03becc8ac15611cfcc421c977a5ba4d654560935707885 23a4ba557689aa7f7 1.05MB / 1.87MB                                   9.0s
 => => sha256:15e759724ff67f262e38bb7c070af9d0b84f959f9b37fa966f68bf2f881a4b62 627B / 627B                                        1.1s
 => => sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153 3.86MB / 3.86MB                                    4.9s
 => => extracting sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153                                         0.3s
 => => extracting sha256:f03becc8ac15611cfcc421c977a5ba4d654560935707885 23a4ba557689aa7f7                                        0.3s
 => => extracting sha256:15e759724ff67f262e38bb7c070af9d0b84f959f9b37fa966f68bf2f881a4b62                                         0.0s
 => => extracting sha256:ff9f59a6a62e9e9f29d7a84fb18865b45664d3f0d061eff7548bd61746dd101c                                         0.0s
 => => extracting sha256:a71873b303e8d75170b7ced2725b01b3ae15ad76f0d4eef16a49335821b6a0ef                                         0.0s
 => => extracting sha256:34dfdd2ef1f920d0054dde2fc09ddc83ff8e71d05fadb79e2cab6e6234596f0a                                         0.0s
 => => extracting sha256:c8a2fa3a88d244a3f32dcbc9c1f7649c662661a28c624198ada43aa0b7598e7f                                         0.0s
 => => extracting sha256:1165b869c51a1a0747d78cec8fab96c30156a979e51ecf2f91aa792e557d94a4                                         1.3s
 => [2/2] COPY dist/ /usr/share/nginx/html                                                                                         0.4s
 => exporting to image                                                                                                             1.3s
 => => exporting layers                                                                                                            0.9s
 => => exporting manifest sha256:a20b1264215b8116df395722c86ca48df9176e04923fa4085c61505947f22119                                  0.0s
 => => exporting config sha256:f72c83f122ab02aa7087821e37f285e9462777885e39a6967dce681c09a29d0b                                    0.0s
 => => exporting attestation manifest sha256:863276da11d8b7940e9747250416ba548ca50930b100d58c2fd77d7b6835837b                      0.0s
 => => exporting manifest list sha256:079dd9240f80f4db722c2de060178839bc99acdf33aa192a129f9d2fdb7b1c6e                             0.0s
 => => naming to docker.io/library/trend-app:latest                                                                                0.0s
 => => unpacking to docker.io/library/trend-app:latest                                                                             0.2s
root@ubuntu:~/Trend# docker run -d -p 3000:80 --name trend trend-app
96ff1efa22381dc51e53527da896ab5c0713498c595ccc3839373db9c5b6b9f0
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint trend (94f9b8bc72c89b8a
4a3579faa71bcb29afda59f1f443ac62f529083875bcd98b): failed to bind host port 0.0.0.0:3000/tcp: address already in use

Run 'docker run --help' for more information
root@ubuntu:~/Trend# # Kill whatever is using port 3000
kill $(lsof -t -i:3000)
root@ubuntu:~/Trend# docker rm trend
docker run -d -p 3000:80 --name trend trend-app
docker ps
trend
fb52a7987bc5b68da3196f3c642604a3101abb21a2e7b36a0afe87864243c6c2
CONTAINER ID   IMAGE       COMMAND                  CREATED         STATUS        PORTS                                     NAMES
fb52a7987bc5   trend-app   "/docker-entrypoint.…"   2 seconds ago   Up 1 second   0.0.0.0:3000->80/tcp, [::]:3000->80/tcp   trend
```




```
root@ubuntu:~/Trend# docker tag trend-app abinayacould/trend-app:latest
docker push abinayacould/trend-app:latest
The push refers to repository [docker.io/abinayacould/trend-app]
883ac774e4c2: Pushed
3ee02d07b7a3: Pushed
f03becc8ac15: Pushed
15e759724ff6: Pushed
ff9f59a6a62e: Pushed
c8a2fa3a88d2: Pushed
1165b869c51a: Pushed
589002ba0eae: Pushed
a71873b303e8: Pushed
34dfdd2ef1f9: Pushed
latest: digest: sha256:079dd9240f80f4db722c2de060178839bc99acdf33aa192a129f9d2fdb7b1c6e size: 856
root@ubuntu:~/Trend#
```

ubuntu | 2% | 0.63 GB / 3.78 GB | 0.01 Mb/s | 0.00 Mb/s | 223 min | abi (x3) | /: 37% | /boot: 7%

ort MobaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net



Part 3 — Terraform Infrastructure

Terraform was used to provision the complete AWS infrastructure as code. A main.tf file defines all resources including VPC, public subnet, internet gateway, route tables, security group, IAM role, and an EC2 instance with Jenkins auto-installed via user_data.

Resources provisioned:

- VPC — trend-vpc (CIDR: 10.0.0.0/16)
- Public Subnet — us-west-2a
- Internet Gateway + Route Table
- Security Group — ports 22, 80, 3000, 8080
- IAM Role + Instance Profile for EC2
- EC2 instance (t3.micro) with Jenkins installed via user_data

```
root@ubuntu:~/terraform# cat  main.tf
provider "aws" {
  region = "us-west-2"
}

# VPC
resource "aws_vpc" "trend_vpc" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags = {
    Name = "trend-vpc"
  }
}

# Subnet
resource "aws_subnet" "trend_subnet" {
  vpc_id                  = aws_vpc.trend_vpc.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "us-west-2a"
  map_public_ip_on_launch = true
  tags = {
    Name = "trend-subnet"
  }
}

# Internet Gateway
resource "aws_internet_gateway" "trend_igw" {
  vpc_id = aws_vpc.trend_vpc.id
  tags = {
    Name = "trend-igw"
  }
}

# Route Table
resource "aws_route_table" "trend_rt" {
  vpc_id = aws_vpc.trend_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.trend_igw.id
  }
```

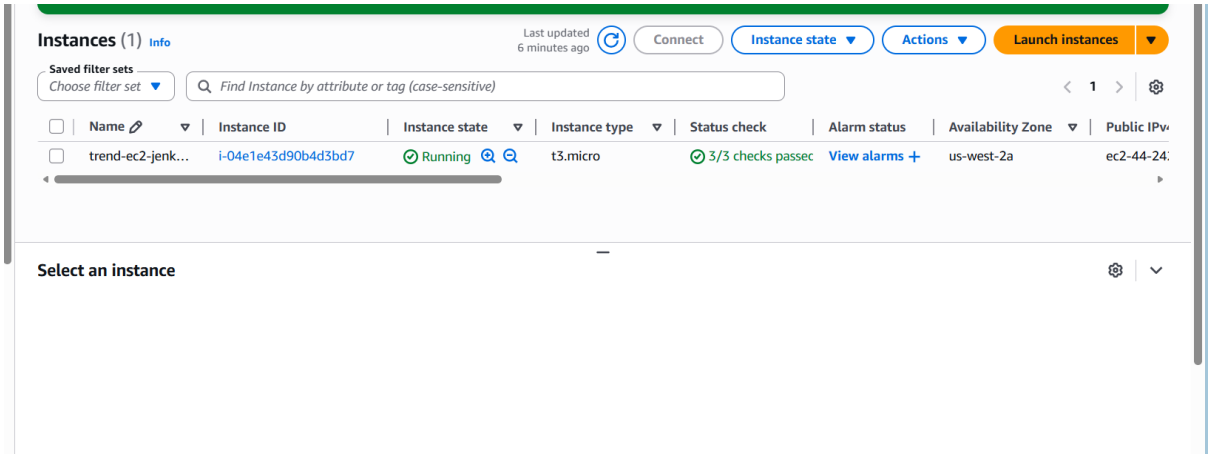
```
  }
}

# Route Table Association
resource "aws_route_table_association" "trend_rta" {
  subnet_id      = aws_subnet.trend_subnet.id
  route_table_id = aws_route_table.trend_rt.id
}

# Security Group
resource "aws_security_group" "trend_sg" {
  name        = "trend-sg"
  description = "Allow SSH, HTTP, Jenkins"
  vpc_id      = aws_vpc.trend_vpc.id

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port   = 3000
    to_port     = 3000
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
```

```
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = {
    Name = "trend-sg"
  }
}

# IAM Role for EC2
resource "aws_iam_role" "trend_ec2_role" {
  name = "trend-ec2-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action    = "sts:AssumeRole"
      Effect    = "Allow"
      Principal = { Service = "ec2.amazonaws.com" }
    }]
  })
}

# IAM Instance Profile
resource "aws_iam_instance_profile" "trend_profile" {
  name = "trend-ec2-profile"
  role = aws_iam_role.trend_ec2_role.name
}

# EC2 Instance with Jenkins
resource "aws_instance" "trend_ec2" {
  ami                    = "ami-0c55b159cbfafe1f0"
  instance_type          = "t2.medium"
  subnet_id              = aws_subnet.trend_subnet.id
  vpc_security_group_ids = [aws_security_group.trend_sg.id]
  iam_instance_profile   = aws_iam_instance_profile.trend_profile.name
```

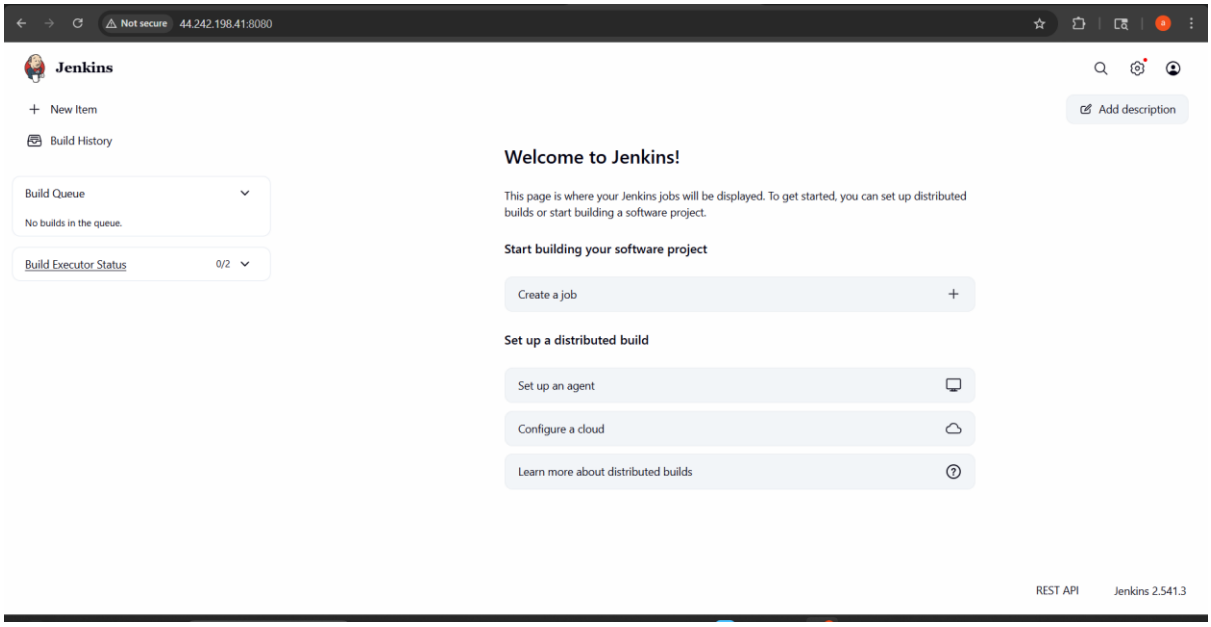
```
  })
}

# IAM Instance Profile
resource "aws_iam_instance_profile" "trend_profile" {
  name = "trend-ec2-profile"
  role = aws_iam_role.trend_ec2_role.name
}

# EC2 Instance with Jenkins
resource "aws_instance" "trend_ec2" {
  ami                    = "ami-0c55b159cbfafe1f0"
  instance_type          = "t2.medium"
  subnet_id              = aws_subnet.trend_subnet.id
  vpc_security_group_ids = [aws_security_group.trend_sg.id]
  iam_instance_profile   = aws_iam_instance_profile.trend_profile.name
  key_name               = "your-key-pair-name"

  user_data = <<-EOF
    #!/bin/bash
    sudo apt update -y
    sudo apt install -y openjdk-17-jdk
    curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee /usr/share/keyrings/jenkins-keyring.asc
    echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/ | sudo tee /etc/apt/sources.list.d/jenkins.list
    sudo apt update -y
    sudo apt install -y jenkins
    sudo systemctl start jenkins
    sudo systemctl enable jenkins
  EOF

  tags = {
    Name = "trend-ec2-jenkins"
  }
}

# Output
output "ec2_public_ip" {
  value = aws_instance.trend_ec2.public_ip
}
root@ubuntu:~/terraform#
```

```
root@ubuntu:~/terraform# terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.40.0...
- Installed hashicorp/aws v6.40.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
root@ubuntu:~/terraform#
```

```
Last login: Sun Apr 12 12:37:44 2026 from 192.168.158.1
abi@ubuntu:~$ sudo su -
[sudo] password for abi:
root@ubuntu:~# cd terraform
root@ubuntu:~/terraform# aws ec2 describe-instances \
  --instance-ids i-04e1e43d90b4d3bd7 \
  --query 'Reservations[0].Instances[0].State.Name' \
  --output text \
  --region us-west-2
running
root@ubuntu:~/terraform# terraform output ec2_public_ip
"54.244.134.170"
root@ubuntu:~/terraform#
```




```
Saving to: 'jenkins.war'

jenkins.war                         100%[===================================================================================>]  91.80M  107MB/s    in 0.9s

2026-04-13 02:41:15 (107 MB/s) - 'jenkins.war' saved [96262515/96262515]

Created symlink /etc/systemd/system/multi-user.target.wants/jenkins.service → /etc/systemd/system/jenkins.service.
● jenkins.service - Jenkins
     Loaded: loaded (/etc/systemd/system/jenkins.service; enabled; vendor preset: enabled)
     Active: active (running) since Mon 2026-04-13 02:41:16 UTC; 273ms ago
   Main PID: 3869 (java)
      Tasks: 11 (limit: 1071)
     Memory: 42.2M
        CPU: 64ms
     CGroup: /system.slice/jenkins.service
             └─3869 /usr/bin/java -jar /opt/jenkins/jenkins.war --httpPort=8080

Apr 13 02:41:16 ip-10-0-1-150 systemd[1]: Started Jenkins.
```



Part 4 — Kubernetes (AWS EKS)

An AWS EKS cluster named trend-cluster was provisioned using eksctl in the us-west-2 region. Kubernetes deployment and service YAML files were created. The application was deployed as 2 replicas and exposed via an AWS LoadBalancer.

Steps completed:

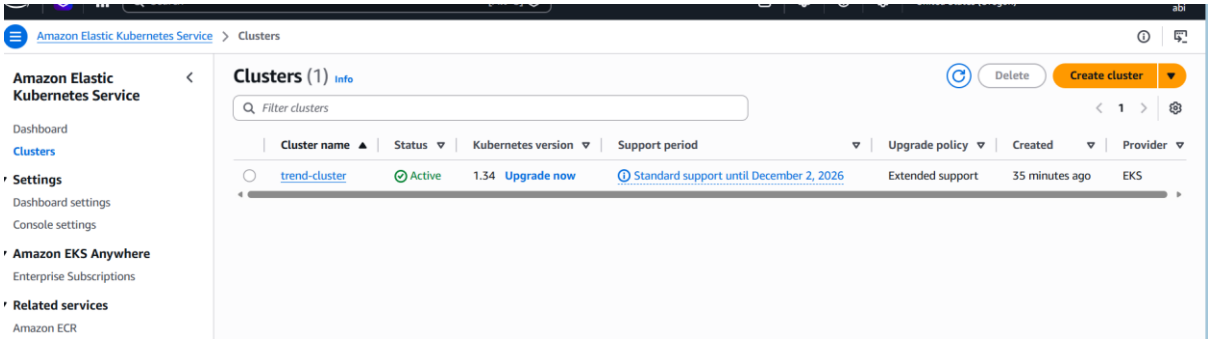
- EKS cluster created — trend-cluster (us-west-2)
- Node group: trend-nodes — 4 x t3.micro nodes
- deployment.yaml — 2 replicas of trend-app
- service.yaml — LoadBalancer type on port 80
- Pods deployed and verified Running
- LoadBalancer external URL confirmed working

```
ubuntu@ip-10-0-1-150:~$ sudo cat /home/jenkins/.jenkins/secrets/initialAdminPassword
2023e476c6214567a6fa73f2112289da
ubuntu@ip-10-0-1-150:~$
ubuntu@ip-10-0-1-150:~$ curl -LO "https://dl.k8s.io/release/$(curl -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 55.8M  100 55.8M    0     0  86.5M      0 --:--:-- --:--:-- --:--:-- 86.6M
Client Version: v1.35.3
Kustomize Version: v5.7.1
ubuntu@ip-10-0-1-150:~$ curl -sLO "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_Linux_amd64.tar.gz"
tar -xzf eksctl_Linux_amd64.tar.gz
sudo mv eksctl /usr/local/bin/
eksctl version
0.225.0
ubuntu@ip-10-0-1-150:~$
```

```
  inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/LICENSE
You can now run: /usr/local/bin/aws --version
aws-cli/2.34.29 Python/3.14.3 Linux/6.8.0-1051-aws exe/x86_64.ubuntu.22
ubuntu@ip-10-0-1-150:~$ aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.

AWS Access Key ID [None]: AKIA4HD2FWIBAAOZZJDK
AWS Secret Access Key [None]: PG/emxs+drM2CGP/HEUsgEHIK+jxmhEvT19oskxW
Default region name [None]: us-west-2
Default output format [None]: json
ubuntu@ip-10-0-1-150:~$ kubectl version --client
eksctl version
Client Version: v1.35.3
Kustomize Version: v5.7.1
0.225.0
ubuntu@ip-10-0-1-150:~$
ubuntu  1%  0.63 GB / 3.78 GB  0.01 Mb/s  0.00 Mb/s  6 hours  abi (x3)  /: 53%  /boot: 7%
```

EKS

```
2026-04-13 03:32:19 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-04-13 03:33:19 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-04-13 03:34:19 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-04-13 03:35:19 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-04-13 03:36:19 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-04-13 03:37:19 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-04-13 03:38:19 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-04-13 03:39:19 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-cluster"
2026-04-13 03:39:20 [!]  recommended policies were found for "vpc-cni" addon, but since OIDC is disabled on the cluster, eksctl cannot configure the requested
 permissions; the recommended way to provide IAM permissions for "vpc-cni" addon is via pod identity associations; after addon creation is completed, add all
recommended policies to the config file, under `addon.PodIdentityAssociations`, and run `eksctl update addon`
2026-04-13 03:39:20 [ℹ]  creating addon: vpc-cni
2026-04-13 03:39:20 [ℹ]  successfully created addon: vpc-cni
2026-04-13 03:39:21 [ℹ]  creating addon: kube-proxy
2026-04-13 03:39:21 [ℹ]  successfully created addon: kube-proxy
2026-04-13 03:39:21 [ℹ]  creating addon: coredns
2026-04-13 03:39:22 [ℹ]  successfully created addon: coredns
2026-04-13 03:41:22 [ℹ]  building managed nodegroup stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-04-13 03:41:22 [ℹ]  deploying stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-04-13 03:41:22 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-04-13 03:41:52 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-04-13 03:42:27 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-04-13 03:44:07 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-04-13 03:44:07 [ℹ]  waiting for the control plane to become ready
2026-04-13 03:44:09 [✔]  saved kubeconfig as "/home/ubuntu/.kube/config"
2026-04-13 03:44:09 [ℹ]  no tasks
2026-04-13 03:44:09 [✔]  all EKS cluster resources for "trend-cluster" have been created
2026-04-13 03:44:09 [ℹ]  nodegroup "trend-nodes" has 2 node(s)
2026-04-13 03:44:09 [ℹ]  node "ip-192-168-25-81.us-west-2.compute.internal" is ready
2026-04-13 03:44:09 [ℹ]  node "ip-192-168-85-119.us-west-2.compute.internal" is ready
2026-04-13 03:44:09 [ℹ]  waiting for at least 1 node(s) to become ready in "trend-nodes"
2026-04-13 03:44:09 [ℹ]  nodegroup "trend-nodes" has 2 node(s)
2026-04-13 03:44:09 [ℹ]  node "ip-192-168-25-81.us-west-2.compute.internal" is ready
2026-04-13 03:44:09 [ℹ]  node "ip-192-168-85-119.us-west-2.compute.internal" is ready
2026-04-13 03:44:09 [✔]  created 1 managed nodegroup(s) in cluster "trend-cluster"
2026-04-13 03:44:09 [ℹ]  creating addon: metrics-server
2026-04-13 03:44:10 [ℹ]  successfully created addon: metrics-server
2026-04-13 03:44:11 [ℹ]  kubectl command should work with "/home/ubuntu/.kube/config", try 'kubectl get nodes'
2026-04-13 03:44:11 [✔]  EKS cluster "trend-cluster" in "us-west-2" region is ready
ubuntu@ip-10-0-1-150:~$
```




```
2026-04-13 03:44:11 [✔]  EKS cluster "trend-cluster" in "us-west-2" region is ready
ubuntu@ip-10-0-1-150:~$ kubectl get nodes
NAME                                           STATUS   ROLES    AGE   VERSION
ip-192-168-25-81.us-west-2.compute.internal    Ready    <none>   27m   v1.34.4-eks-f69f56f
ip-192-168-85-119.us-west-2.compute.internal   Ready    <none>   27m   v1.34.4-eks-f69f56f
ubuntu@ip-10-0-1-150:~$
```

deployment and service YAML files.

```
2026-04-13 03:44:11 [✔]  EKS cluster "trend-cluster" in "us-west-2" region is ready
ubuntu@ip-10-0-1-150:~$ kubectl get nodes
NAME                                           STATUS   ROLES    AGE   VERSION
ip-192-168-25-81.us-west-2.compute.internal    Ready    <none>   27m   v1.34.4-eks-f69f56f
ip-192-168-85-119.us-west-2.compute.internal   Ready    <none>   27m   v1.34.4-eks-f69f56f
ubuntu@ip-10-0-1-150:~$ mkdir -p ~/k8s
cd ~/k8s
ubuntu@ip-10-0-1-150:~/k8s$ nano deployment.yaml
ubuntu@ip-10-0-1-150:~/k8s$ cat deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: trend-deployment
  labels:
    app: trend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: trend
  template:
    metadata:
      labels:
        app: trend
    spec:
      containers:
      - name: trend
        image: abinayacould/trend-app:latest
        ports:
        - containerPort: 80
ubuntu@ip-10-0-1-150:~/k8s$
```

```
ubuntu@ip-10-0-1-150:~/k8s$ nano deployment.yaml
ubuntu@ip-10-0-1-150:~/k8s$ cat deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: trend-deployment
  labels:
    app: trend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: trend
  template:
    metadata:
      labels:
        app: trend
    spec:
      containers:
      - name: trend
        image: abinayacould/trend-app:latest
        ports:
        - containerPort: 80
ubuntu@ip-10-0-1-150:~/k8s$ nano service.yaml
ubuntu@ip-10-0-1-150:~/k8s$ cat service.yaml
apiVersion: v1
kind: Service
metadata:
  name: trend-service
spec:
  selector:
    app: trend
  type: LoadBalancer
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
ubuntu@ip-10-0-1-150:~/k8s$
```

Deployed to EKS

```
    targetPort: 80
ubuntu@ip-10-0-1-150:~/k8s$ kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get pods
kubectl get svc
deployment.apps/trend-deployment created
service/trend-service created
NAME                                 READY   STATUS    RESTARTS   AGE
trend-deployment-7bdc9d59f4-2pmcr    0/1     Pending   0          2s
trend-deployment-7bdc9d59f4-brhzc    0/1     Pending   0          2s
NAME            TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
kubernetes      ClusterIP      10.100.0.1      <none>        443/TCP        41m
trend-service   LoadBalancer   10.100.154.170  <pending>     80:30420/TCP   2s
ubuntu@ip-10-0-1-150:~/k8s$
```

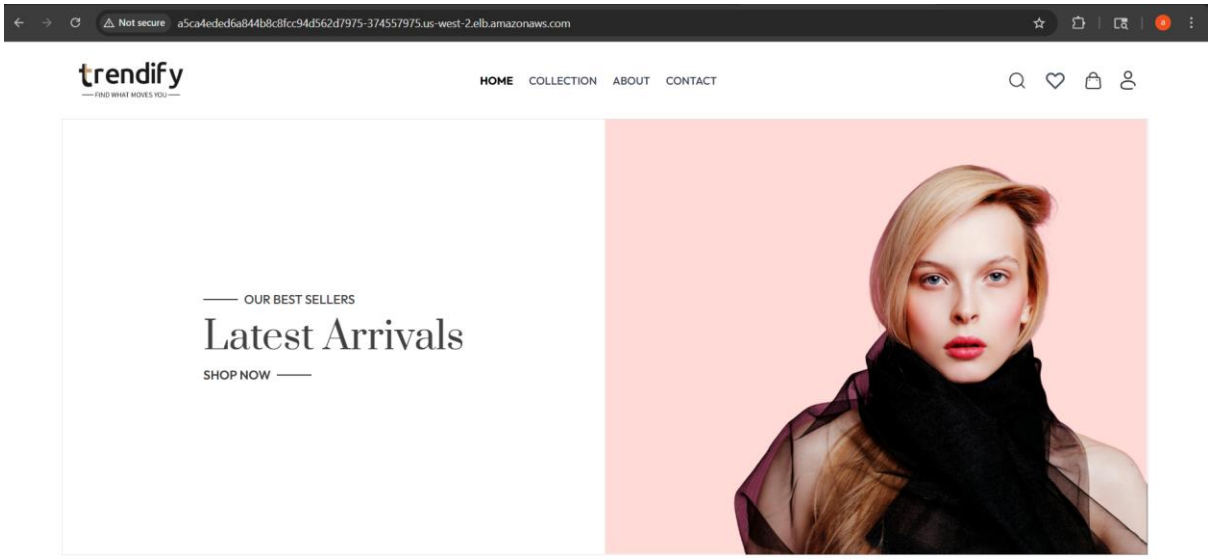
Remote monitoring

Follow terminal folder

UNREGISTERED VERSION - Please support MobaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

```
ubuntu@ip-10-0-1-150:~/k8s$ kubectl get nodes
kubectl get pods
NAME                                          STATUS   ROLES    AGE   VERSION
ip-192-168-25-81.us-west-2.compute.internal   Ready    <none>   61m   v1.34.4-eks-f69f56f
ip-192-168-47-198.us-west-2.compute.internal  Ready    <none>   83s   v1.34.4-eks-f69f56f
ip-192-168-85-119.us-west-2.compute.internal  Ready    <none>   61m   v1.34.4-eks-f69f56f
NAME                                 READY   STATUS    RESTARTS   AGE
trend-deployment-7bdc9d59f4-2pmcr    1/1     Running   0          25m
trend-deployment-7bdc9d59f4-brhzc    1/1     Running   0          25m
ubuntu@ip-10-0-1-150:~/k8s$ kubectl get svc trend-service
NAME            TYPE           CLUSTER-IP      EXTERNAL-IP                                                              PORT(S)        AGE
trend-service   LoadBalancer   10.100.154.170  a5ca4eded6a844b8c8fcc94d562d7975-374557975.us-west-2.elb.amazonaws.com   80:30420/TCP   26m
ubuntu@ip-10-0-1-150:~/k8s$
```

LoadBalancer URL working



Part 5— Jenkins CI/CD Pipeline

Jenkins was configured on the Terraform-provisioned EC2 instance. A declarative pipeline was created with GitHub webhook integration for automatic builds on every push. The pipeline covers 4 stages: clone, build, push, and deploy.

Pipeline stages:

- Stage 1 — Clone Repository from GitHub
- Stage 2 — Build Docker Image
- Stage 3 — Push image to Docker Hub
- Stage 4 — Deploy to EKS using kubectl

Configuration:

- Plugins installed: Docker, Docker Pipeline, Git, Pipeline, Kubernetes CLI
- Credentials configured: GitHub token, Docker Hub password
- GitHub webhook configured for automatic trigger on push
- kubeconfig and AWS credentials configured for Jenkins user

```
root@ubuntu:~/Trend# cat Jenkinsfile
pipeline {
    agent any

    environment {
        DOCKER_IMAGE = 'abinayacould/trend-app'
        DOCKER_TAG = 'latest'
    }

    stages {
        stage('Clone Repository') {
            steps {
                git credentialsId: 'github-creds',
                    url: 'https://github.com/abinayanachimuthu123-beep/Trend.git',
                    branch: 'main'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t ${DOCKER_IMAGE}:${DOCKER_TAG} .'
            }
        }

        stage('Push to Docker Hub') {
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'dockerhub-creds',
                    usernameVariable: 'DOCKER_USER',
                    passwordVariable: 'DOCKER_PASS'
                )]) {
                    sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin'
                    sh 'docker push ${DOCKER_IMAGE}:${DOCKER_TAG}'
                }
            }
```

```
        stage('Build Docker Image') {
            steps {
                sh 'docker build -t ${DOCKER_IMAGE}:${DOCKER_TAG} .'
            }
        }

        stage('Push to Docker Hub') {
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'dockerhub-creds',
                    usernameVariable: 'DOCKER_USER',
                    passwordVariable: 'DOCKER_PASS'
                )]) {
                    sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin'
                    sh 'docker push ${DOCKER_IMAGE}:${DOCKER_TAG}'
                }
            }
        }

        stage('Deploy to EKS') {
            steps {
                sh 'kubectl apply -f k8s/deployment.yaml'
                sh 'kubectl apply -f k8s/service.yaml'
                sh 'kubectl rollout status deployment/trend-deployment'
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully!'
        }
        failure {
            echo 'Pipeline failed!'
        }
    }
}
root@ubuntu:~/Trend#
```

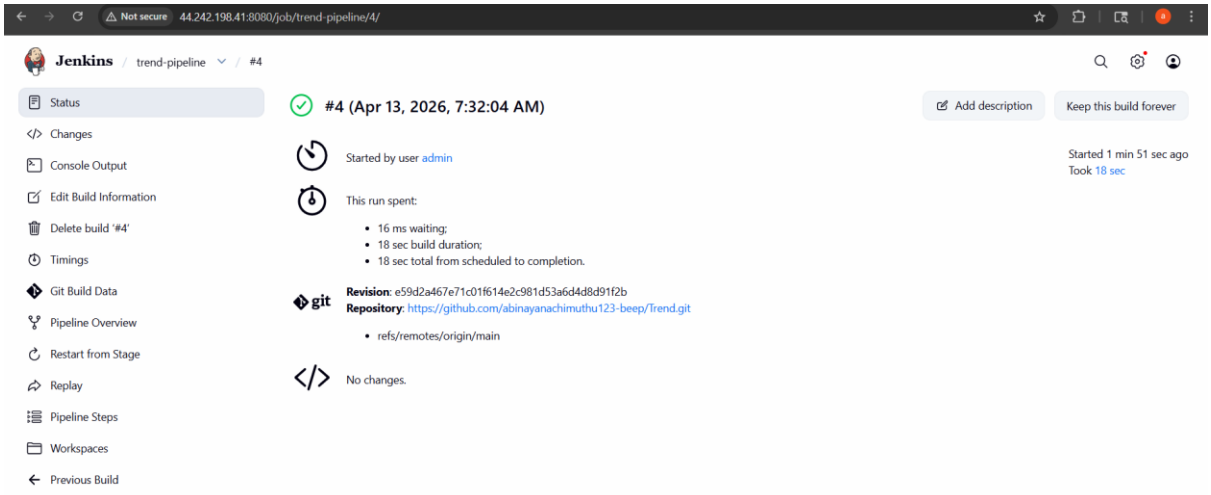
```
    }
}
root@ubuntu:~/Trend# mkdir -p ~/Trend/k8s
cp ~/k8s/deployment.yaml ~/Trend/k8s/
cp ~/k8s/service.yaml ~/Trend/k8s/
cp: cannot stat '/root/k8s/deployment.yaml': No such file or directory
cp: cannot stat '/root/k8s/service.yaml': No such file or directory
root@ubuntu:~/Trend# mkdir -p ~/Trend/k8s
nano ~/Trend/k8s/deployment.yaml
root@ubuntu:~/Trend# nano ~/Trend/k8s/service.yaml
root@ubuntu:~/Trend# ls ~/Trend/k8s/
deployment.yaml  service.yaml
root@ubuntu:~/Trend#
```

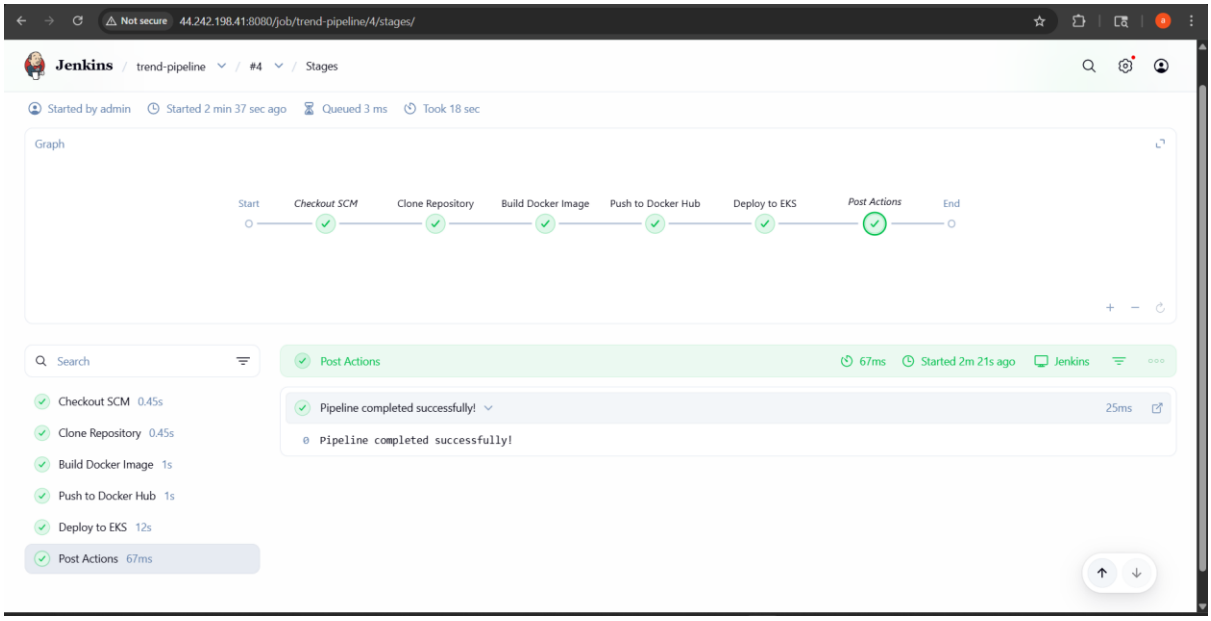
```
root@ubuntu:~/Trend# cat ~/Trend/k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: trend-deployment
  labels:
    app: trend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: trend
  template:
    metadata:
      labels:
        app: trend
    spec:
      containers:
      - name: trend
        image: abinayacould/trend-app:latest
        ports:
        - containerPort: 80
root@ubuntu:~/Trend# cat ~/Trend/k8s/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: trend-service
spec:
  selector:
    app: trend
  type: LoadBalancer
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
root@ubuntu:~/Trend#
```

```
kubectl get nodes
Added new context arn:aws:eks:us-west-2:839922332162:cluster/trend-cluster to /home/ubuntu/.kube/config
NAME                                          STATUS   ROLES    AGE     VERSION
ip-192-168-25-81.us-west-2.compute.internal   Ready    <none>   3h47m   v1.34.4-eks-f69f56f
ip-192-168-47-198.us-west-2.compute.internal  Ready    <none>   167m    v1.34.4-eks-f69f56f
ip-192-168-85-119.us-west-2.compute.internal  Ready    <none>   3h47m   v1.34.4-eks-f69f56f
ubuntu@ip-10-0-1-150:~$ sudo cp ~/.kube/config /home/jenkins/.kube/config
sudo chown -R jenkins:jenkins /home/jenkins/.kube
sudo chmod 600 /home/jenkins/.kube/config
ubuntu@ip-10-0-1-150:~$ sudo mkdir -p /home/jenkins/.aws
sudo cp ~/.aws/credentials /home/jenkins/.aws/
sudo cp ~/.aws/config /home/jenkins/.aws/
sudo chown -R jenkins:jenkins /home/jenkins/.aws
ubuntu@ip-10-0-1-150:~$ sudo -u jenkins kubectl get nodes
NAME                                          STATUS   ROLES    AGE     VERSION
ip-192-168-25-81.us-west-2.compute.internal   Ready    <none>   3h48m   v1.34.4-eks-f69f56f
ip-192-168-47-198.us-west-2.compute.internal  Ready    <none>   168m    v1.34.4-eks-f69f56f
ip-192-168-85-119.us-west-2.compute.internal  Ready    <none>   3h48m   v1.34.4-eks-f69f56f
ubuntu@ip-10-0-1-150:~$
```

Pipeline success:

```
root@ubuntu:~/Trend# cd ~/Trend

git add .

git config user.email "abinayanachimuthu123@gmail.com"

git config user.name "abinayanachimuthu123-beep"

git commit -m "Add Jenkinsfile and k8s manifests"

git push origin main
[main e59d2a4] Add Jenkinsfile and k8s manifests
 5 files changed, 97 insertions(+)
 create mode 100644 Dockerfile
 create mode 100644 Jenkinsfile
 create mode 100644 k8s/deployment.yaml
 create mode 100644 k8s/service.yaml
 create mode 100644 package-lock.json
Username for 'https://github.com': abinayanachimuthu123-beep
Password for 'https://abinayanachimuthu123-beep@github.com':
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 2 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 1.49 KiB | 380.00 KiB/s, done.
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/abinayanachimuthu123-beep/Trend.git
   320282c..e59d2a4  main -> main
root@ubuntu:~/Trend#
```





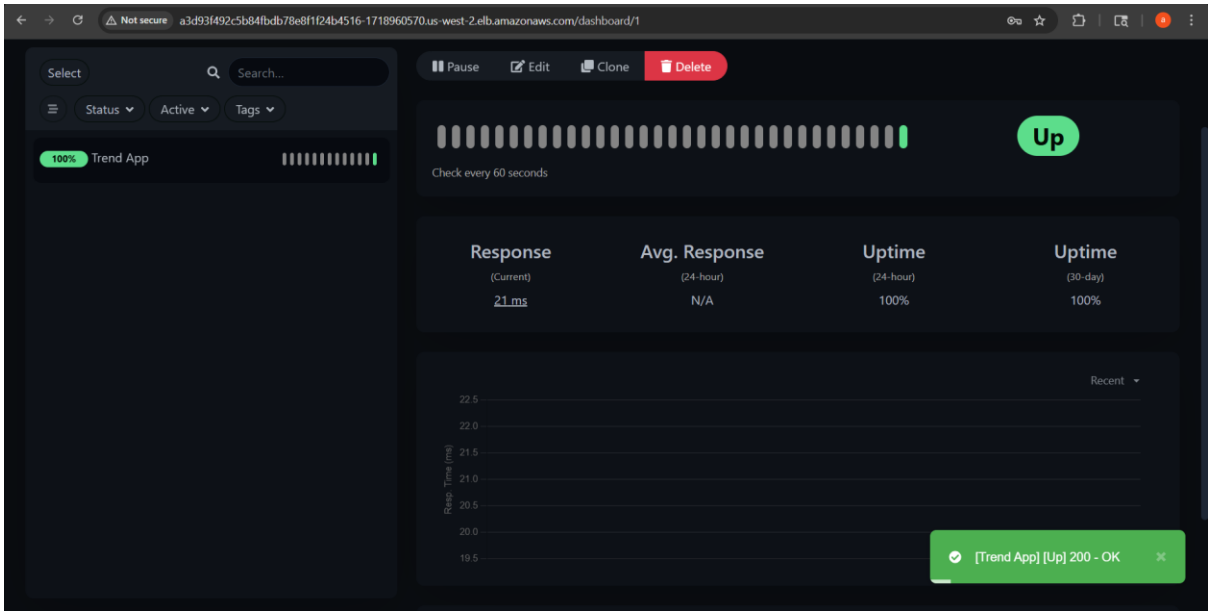
Part 6 — Monitoring

Uptime Kuma was deployed as a Kubernetes pod with a LoadBalancer service for monitoring the health of the Trend application running on EKS. The monitor checks the app every 50 seconds and reports 100% uptime with an average response time of ~19ms.

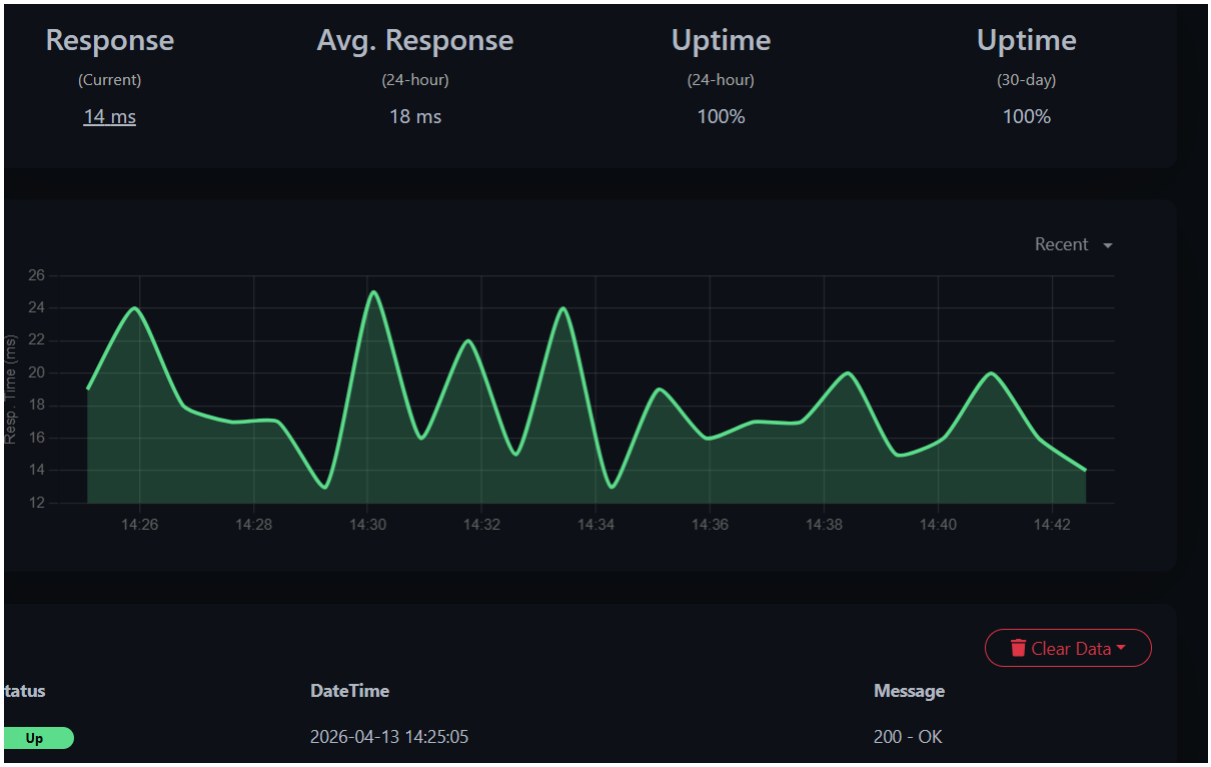
- Uptime Kuma deployed as Kubernetes pod
- Monitor configured to check Trend app LoadBalancer URL
- Uptime: 100% (24-hour and 30-day)
- Average response time: 19ms

HTTP Status: 200 OK

```
ubuntu@ip-10-0-1-150:~$ kubectl get pods
kubectl get svc uptime-kuma-service
NAME                                    READY   STATUS    RESTARTS   AGE
trend-deployment-7bdc9d59f4-2pmcr       1/1     Running   0          4h27m
trend-deployment-7bdc9d59f4-brhzc       1/1     Running   0          4h27m
uptime-kuma-7bf9576cd8-gzcsc            0/1     Pending   0          43s
NAME                 TYPE           CLUSTER-IP       EXTERNAL-IP                                                              PORT(S)        AGE
uptime-kuma-service  LoadBalancer   10.100.141.242   a3d93f492c5b84fbdb78e8f1f24b4516-1718960570.us-west-2.elb.amazonaws.com   80:30358/TCP   44s
ubuntu@ip-10-0-1-150:~$ eksctl scale nodegroup \
  --cluster trend-cluster \
  --name trend-nodes \
  --nodes 4 \
  --nodes-max 5 \
  --region us-west-2
2026-04-13 08:46:49 [ℹ]  scaling nodegroup "trend-nodes" in cluster trend-cluster
2026-04-13 08:46:50 [ℹ]  initiated scaling of nodegroup
2026-04-13 08:46:50 [ℹ]  to see the status of the scaling run `eksctl get nodegroup --cluster trend-cluster --region us-west-2 --name trend-nodes`
ubuntu@ip-10-0-1-150:~$ kubectl get pods
NAME                                    READY   STATUS    RESTARTS   AGE
trend-deployment-7bdc9d59f4-2pmcr       1/1     Running   0          4h29m
trend-deployment-7bdc9d59f4-brhzc       1/1     Running   0          4h29m
uptime-kuma-7bf9576cd8-gzcsc            1/1     Running   0          2m59s
ubuntu@ip-10-0-1-150:~$
```



The below status was taken while destroying the EKS to record the error




```
ubuntu@ip-10-0-1-150:~$ eksctl delete cluster --name trend-cluster --region us-west-2
2026-04-13 09:12:43 [ℹ]  deleting EKS cluster "trend-cluster"
2026-04-13 09:12:43 [ℹ]  will drain 0 unmanaged nodegroup(s) in cluster "trend-cluster"
2026-04-13 09:12:43 [ℹ]  starting parallel draining, max in-flight of 1
2026-04-13 09:12:43 [ℹ]  deleted 0 Fargate profile(s)
2026-04-13 09:12:43 [✓]  kubeconfig has been updated
2026-04-13 09:12:43 [ℹ]  cleaning up AWS load balancers created by Kubernetes objects of Kind Service, Ingress, or Gateway
2026-04-13 09:13:54 [ℹ]
2 sequential tasks: { delete nodegroup "trend-nodes", delete cluster control plane "trend-cluster" [async]
}
2026-04-13 09:13:54 [ℹ]  will delete stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-04-13 09:13:54 [ℹ]  waiting for stack "eksctl-trend-cluster-nodegroup-trend-nodes" to get deleted
2026-04-13 09:13:54 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-04-13 09:14:24 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-04-13 09:15:01 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-04-13 09:15:53 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-04-13 09:17:24 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-04-13 09:19:00 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-04-13 09:20:14 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-04-13 09:21:07 [ℹ]  waiting for CloudFormation stack "eksctl-trend-cluster-nodegroup-trend-nodes"
2026-04-13 09:21:08 [ℹ]  will delete stack "eksctl-trend-cluster-cluster"
2026-04-13 09:21:08 [✓]  all cluster resources were deleted
ubuntu@ip-10-0-1-150:~$
```

Project Fully Complete!

GitHub repo: https://github.com/abinayanachimuthu123-beep/Trend

LoadBalancer ARN → a5ca4eded6a844b8c8fcc94d562d7975-374557975.us-west-2.elb.amazonaws.com
Docker Hub → abinayacould/trend-app